rba.core
Building constraint matrices from RBA data.

Biosys - MAIAGE

March 30, 2026

Contents

1. Overview of RBA constraints **3**
1.1. Important concepts and conventions 3
1.2. Variables 4
1.3. Mathematical formulation of RBA constraints 4

2. Constraint matrices **6**
2.1. Mass conservation (C_1) 6
2.2. Capacity constraints (C_2) 6
2.3. Compartment occupation constraints (C_3) 8

3. Building matrices from XML files **9**

A. Eliminating target variables **12**

1. Overview of RBA constraints

In order to build an RBA system, we need to build three sets of constraints.

(C_1) Mass conservation.

(C_2) Capacity constraints for enzymes and process machines, e.g., ribosomes.

(C_3) Compartment occupation constraints.

- ($C_3 - a$) Occupation constraint for each compartment.
- ($C_3 - b$) Global occupation constraint.

All these constraints are linear (in)equalities. We will write them in matrix form, as it is the best way to summarize all information needed to build a constraint.

1.1. Important concepts and conventions

Production/degradation vector for macromolecules In the following, each macromolecule is described by its **production vector**. It is a column vector containing the metabolites necessary to build it, with a **minus** sign for metabolites consumed and a **plus** sign for byproducts generated. Similarly, we define **degradation vectors** when (macro-)molecules are decomposed into individual constituents by catalytic or spontaneous processes.

Concentration to flux conversion Suppose you have a metabolite at concentration $C = n/V$. At growth-rate μ, we have $dV \simeq \mu V$. The variation of concentration due to dilution is

$$dC = d(n/V) = -n/V^2 dV = -\mu C$$

However when the given species is also subject to degradation, a respective term must be added, representing the dilution by degradation at rate λ (which is zero for stable molecular species). In other words, in order to keep the concentration constant, we need an input metabolite flux of

$$\nu_{in} = \mu C + \lambda C$$

The output metabolic flux however is solely dependent on the degradation:

$$\nu_{out} = \lambda C$$

When a variable is expressed as a concentration, it may be necessary to convert it to a flux by using this relationship. When necessary, we indicate this conversion by inserting a conversion matrix between the main matrix and the variable vector.

Variable representation When we go to full matrix representation in the figures, variables are represented as a single row vector below matrices. This convention clarifies how columns and variables are associated. Note the transpose at the end of the row vector if you are worried about mathematical purity.

1.2. Variables

RBA uses 4 sets of variables:

- $\nu = (\nu_1, \ldots, \nu_R)$: Fluxes through metabolic reactions, where R is the number of reactions.
- $E = (e_1, \ldots, e_E)$: Enzyme concentrations, where E is the number of enzymes.
- $P = (p_1, \ldots, p_P)$: Process machine concentrations, where P is the number of processes.
- $C = (c_1, \ldots, c_C)$: Compartment sizes, where C is the number of cellular compartments.
- $T = (t_1, \ldots, t_T) = (TF, TC)$: Target values, where T is the number of targets. Targets may be expressed as fluxes or concentrations. We note target fluxes TF and target concentrations TC. When necessary, we assume that flux targets are all listed first, i.e., $T = (TF, TC)$.

Targets are production/degradation requirements for the cell to be fully functional (e.g. keeping key metabolites at some given concentration, producing housekeeping proteins, producing/degrading mRNAs). Note that target values may actually be predetermined constant values, while all others are true variables that must be optimized. We will see later how predetermined constant values may be eliminated.

1.3. Mathematical formulation of RBA constraints

Before we group all constraints and variables into a single matrix, here is an overview of the 3 family of constraints used in RBA models.

(C_1) **Mass conservation**

$$\overbrace{\underbrace{S\nu}_{\text{metabolic flux}} + \underbrace{(\mu+\lambda)C_E^+E + (\mu+\lambda)C_P^+P + \lambda C_E^-E + \lambda C_P^-P}_{\substack{\text{precursors used/byproducts generated by producing new molecules } (C_{\dots}^+). \text{ And metabolites produced and consumed during degradation } (C_{\dots}^-).}}}^{\text{'Variable' terms}}$$
$$+\overbrace{\underbrace{(\mu+\lambda)C_{TC}^+TC + \lambda C_{TC}^-TC + C_{TF}TF}_{\substack{\text{precursors used/byproducts generated by producing new molecules } (C_{\dots}^+). \text{ And metabolites produced and consumed during degradation } (C_{\dots}^-).}}}^{\text{Constant terms}} = 0 \tag{1.1}$$

where S is the stoichiometry matrix of the metabolism, with metabolites as rows and reactions as columns. $C_{.}$ are composition matrices where every column is the composition of one of the molecules, e.g. $(C_E)_i$ is the composition of the ith enzyme (see definition of composition vector above). Note the usage of μ along with E, P and TC as way of converting concentrations into fluxes.

(C_2) **Capacity constraints**

$$\text{diag}\left(k_E^{backward}\right)E \leq \nu \leq \text{diag}\left(k_E^{forward}\right)E$$

where $k_E^{backward}$ is a vector containing the backward catalytic constants of enzymes.

$$\begin{aligned}(\mu+\lambda)MC_E^+E + (\mu+\lambda)MC_P^+P + (\mu+\lambda)MC_{TC}^+TC + MC_{TF}TF \\ +\lambda MC_E^-E + \lambda MC_P^-P + \lambda MC_{TC}^-TC + MC_{TF}TF \\ \leq \text{diag}\left(k_P\right)P\end{aligned} \tag{1.2}$$

where k_P is a vector containing the capacities of process machines. $MC_{.}$ are matrices containing machine costs to produce macromolecules, e.g. $(MC_E)_{ij}$ quantifies how much the machine from the ith process is needed to build the jth enzyme. For example, the ribosome (machine of translation process) has a processing capacity of $k_P \simeq 72,000$ amino acid per hour. If an enzyme contains 3 proteins of 300 amino acid each, its machine cost is $MC = 900$.

$(C_3 - a)$ **Compartment occupation constraints**

$$W_EE + W_PP + W_{TC}TC - O_i = 0$$

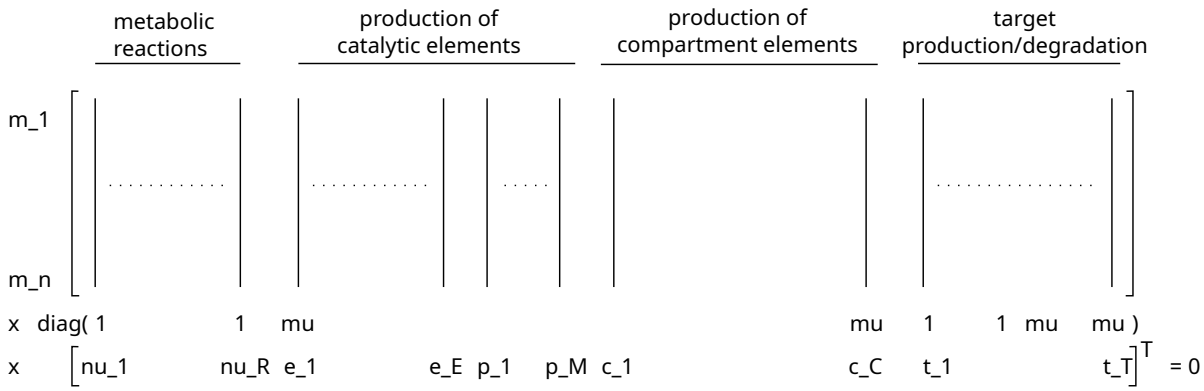

Figure 1: Mass conservation constraint. Bars in the main matrix are metabolic reaction or production/degradation vectors associated with each variable.

where $W_{\cdot}$ are weight matrices, e.g. $(W_E)_{ij}$ is the weight of the jth enzyme in the ith compartement. O_i is a decision variable, representing the occupation of the ith compartment.

$(C_3 - b)$ **Global occupation constraint**

$$\sum_{i=1..C} (O_i) \leq \rho_{tot}$$

where ρ_{tot} represents the total cellular density.

2. Constraint matrices

2.1. Mass conservation (C_1)

The mass conservation constraint is represented by a matrix where rows are metabolites, labelled $M = (m_1, \ldots, m_M)$. Metabolite fluxes must cancel out in order to achive mass conservation. Note that numerous variables are expressed as concentrations and must be converted to fluxes as explained previously.

Figure 1 shows how mass conservation is expressed in matrix formalism. The main matrix contains the metabolic reactions, the production vectors for catalytic elements, the production/degradation vectors for target elements. This matrix is growth-rate independent. The second matrix converts variables that are expressed as concentrations to fluxes. This can also be seen as multiplying the appropriate columns in the main matrix by the growth-rate.

2.2. Capacity constraints (C_2)

The capacity constraintes are represented by a matrix where rows are enzymes or process machineries. Currently, we use two different formalisms for enzymes and process

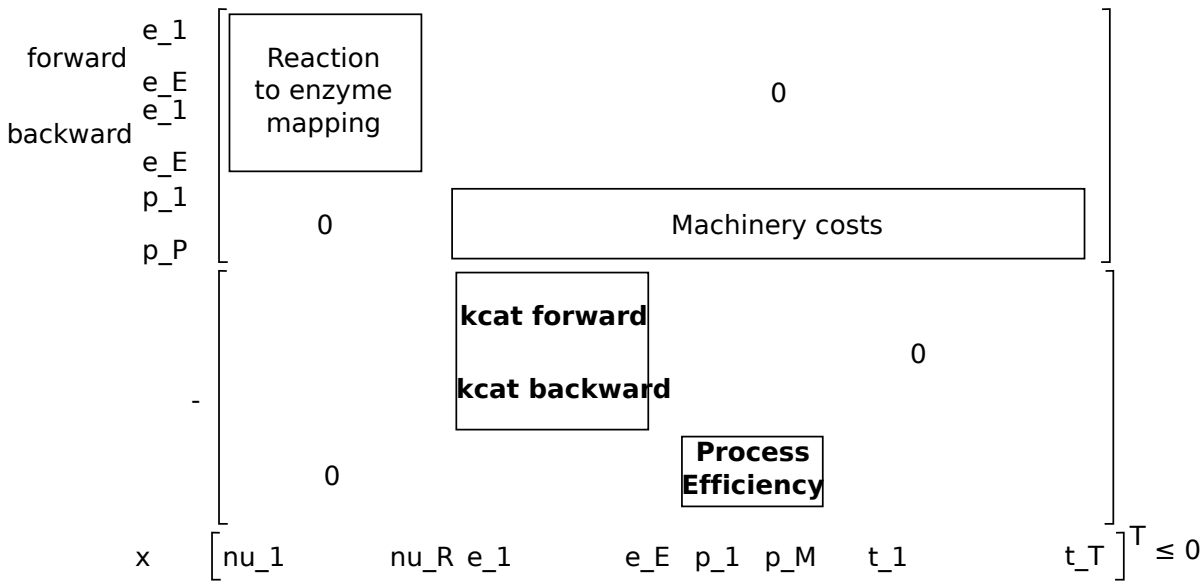

Figure 2: Capacity constraints.

machineries.

Enzymes are associated to exactly one reaction that may be reversible. For every enzyme we have the following constraint:

$$\nu \leq k_{forward} e_i$$

where ν is the flux through the reaction catalyzed by e_i. If the reaction is irreversible, we have the additional constraint

$$\nu \geq -k_{backward} e_i$$

In order to write all constraints, we need a reaction to enzyme mapping. This is represented by a matrix where we have one row per constraint. Each row has a 1 on the column corresponding to the reaction catalyzed, and 0s everywhere else.

Process machineries participate in the synthesis/degradation of several macromolecules (enzymes, machineries and targets). For every target, we have a constraint of the form

$$[machinery_cost].[E, P, T]^T \leq k_{machinery} p_i$$

Every macromolecule has a set of machinery costs associated with it. It tells how much a machinery is used in order to produce/degrade the macromolecule (the cost is often 0).

The final matrices are very sparse (Fig. 2). A first matrix contains the reaction to enzyme mapping and the machinery costs. This matrix is growth-rate independent. A second matrix contains efficiencies, that may depend on growth-rate. Note that we did not need to convert concentrations to fluxes here.

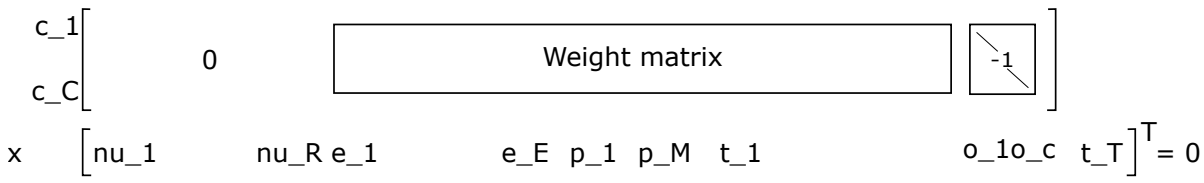

Figure 3: Compartment occupation constraints.

2.3. Compartment occupation constraints (C_3)

Compartment occupation constraints are represented by matrices where rows are compartments, labelled $C = (c_1, \ldots, c_C)$.

Every variable that represents a concentration participates to this constraint. For every macromolecule, we define a weight vector that defines how much volume one molecule occupies in every compartment (in user defined units). By putting these vectors together we get a weight matrix.

The user also defines a vector of maximal weights for every compartment, yielding a simple set of constraints (Fig. 3). Only the right-hand part may contain growth-rate dependent values. Again, no conversion from concentration to flux is needed here.

	Reaction Fluxes (nu)	Enzymes (E)	Process Machineries (P)	Compartment Occupations (C)	Target Concentrations (TC) Target Fluxes (TF)	RHS
Mass conservation	S	muC_E	muC_P	muC_C	muC_TC + C_TF	
Process Capacity		muMC_E	muMC_P - diag(k_P)	muMC_C	muMC_TC + MC_TF	
Enzyme Capacity (forward)	R_to_E	-diag(k^+_E)				
Enzyme Capacity (backward)	-R_to_E	-diag(k^-_E)				
Density Constraints		W_E	W_P	-diag(1-f_HK)	W_TC	

Blocks needed:
S: stoichiometry matrix.
C_.: composition (production or degradation) matrix
MC_.: machinery costs
W_.: weight matrix
k_.: capacity vector
D: density limits for every compartment
R_to_E: reaction to enzyme mapping.
f_HK: Fraction of housekeeping components in compartmlent

Figure 4: Blocks that need to be assembled. There is also a vector of constraint signs that is omitted here (E for equality, L for lower than, G for greater than).

3. Building matrices from XML files

In this file, we briefly describe how matrices are built from files.

Building blocks Figure 4 shows the blocks that are used to build the final matrices.

Stoichiometry matrix The stoichiometry matrix is built from metabolic reactions. External metabolites are removed from the metabolite pool.

Species matrices Figure 5 shows how macromolecules are broken down into matrices describing their composition, machinery cost and weight. In the end, they are merged into a single matrix describing composition, machinery cost and weight of all metabolites and macromolecules.

Enzyme and machinery matrices Figure 6 shows how enzyme and process machinery matrices are built.

Target matrices Targets are either metabolites or macromolecules. Their composition, machinery cost and weight can be extracted as columns from the species matrices.

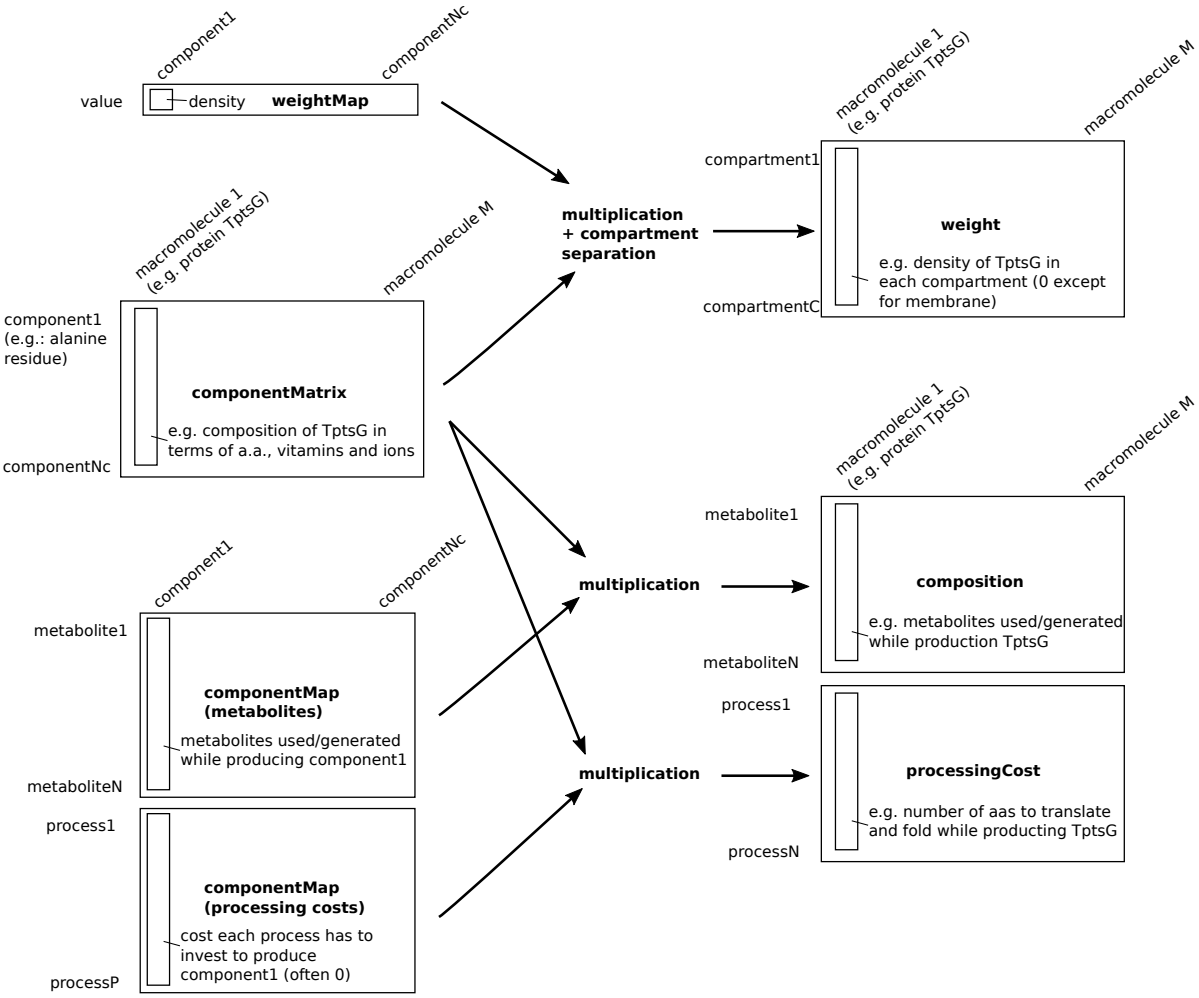

Figure 5: Matrices extracted from macromolecule information. An example is given with proteins but in the end, they contain all macromolecules and all internal metabolites.

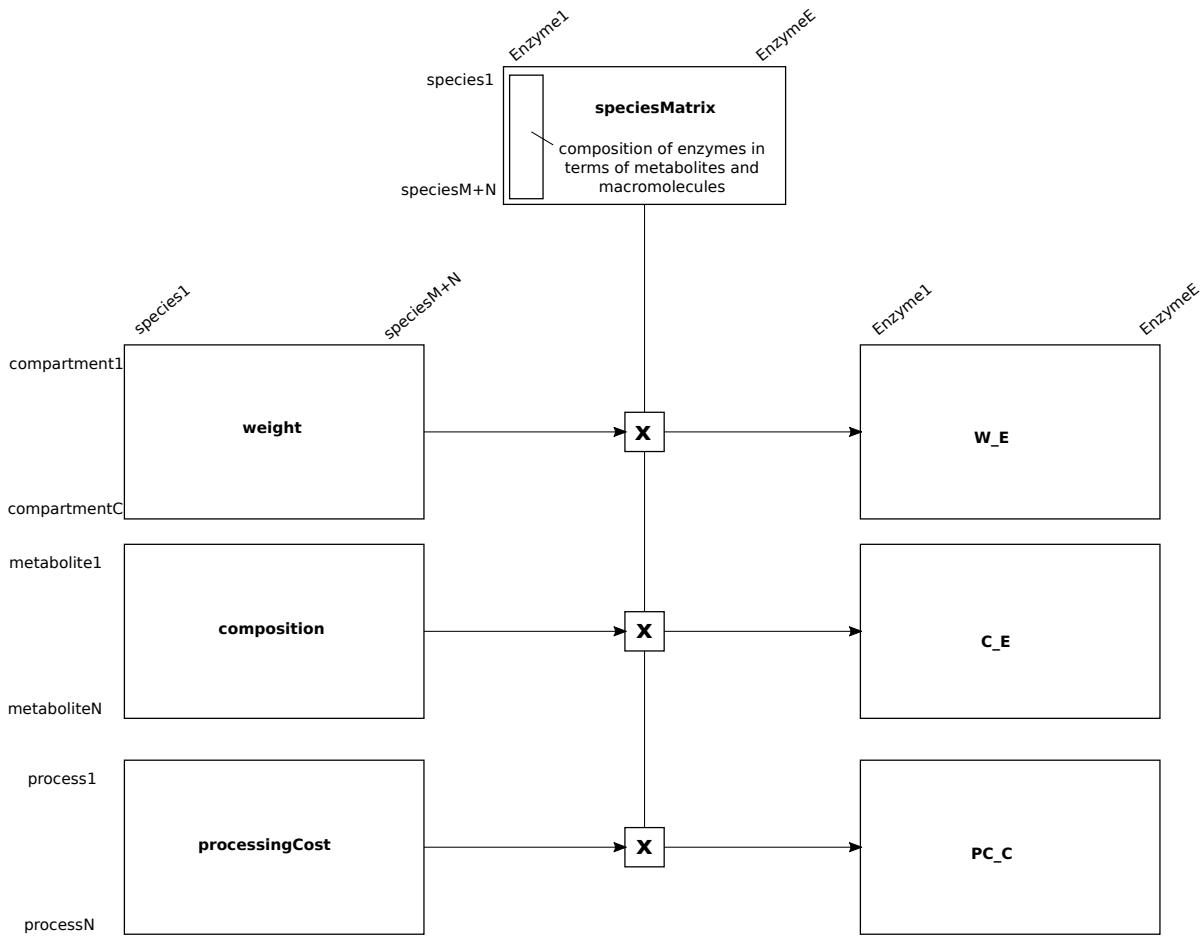

Figure 6: Every machinery can be described by a reaction matrix. Reactants are species (metabolites or macromolecules) needed to build the machinery and products are byproducts of the assembly process. Through matrix multiplication with the species matrices, we can deduce its composition, weight and machinery cost.

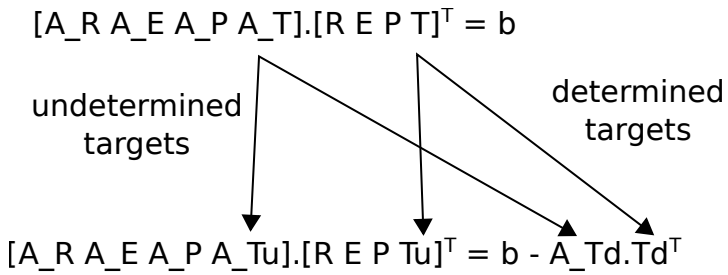

Figure 7: Procedure to reduce matrix size by eliminating targets with predefined values. This yield a smaller A matrix and a slightly more complicated b matrix.

A. Eliminating target variables

In the previous sections, we included all targets as variables. In practice, most variables are predetermined, e.g., the target for concentrations for metabolites is known, it does not need to be optimized. Optimization algorithms will typically eliminate all such variables in a presolving step, but we might want to eliminate them manually in order to reduce matrix sizes.

The procedure is fairly simple: we extract the submatrix corresponding to targets with known values, multiply it by the value vector, and move it to the right-hand side (Fig. 7).